ONLY PERSONS WITH
ID-C5 CARD

T3
MODDING GUIDE

A Beginner's Handbook for Installing and Creating Mods with T3-ModTools.

EDITION 1.0 | AUGUST 2026
Based on T3-ModTools 0.6.0.

T3-ModTools is experimental.
Keep original files and your own work backed up.

Contents

Use the table of contents below in Word or LibreOffice. PDF exports also preserve the heading structure for navigation.

How to Use This Book

This book is written for people who have never made a game mod. It begins with the mental model that makes every later step easier, then walks through installing a proven mod set, extracting assets, creating a first replacement mod, and building more advanced audio, particle, weapon, model, LOD, and animation projects.

The safest learning path is:

1. Read Part I and install the known working mods.
2. Complete the first replacement project in Part III.
3. Choose one workshop in Part IV.
4. Move to Blender or executable patches only after ordinary file replacement feels routine.

The guide uses three evidence labels:

Label	Meaning
Verified	The workflow is implemented by the current tool and has been exercised against the reference game installation.
Recommended	A practical starting point based on the current project, but it still needs testing on the target asset.
Experimental	The feature works at a structural level or in a specific case, but is not a general solution.

T3 is an old game with a small public modding knowledge base. That makes disciplined testing especially important. Change one thing, build, test, record the result, and keep the last known good version.

Scope of the Reference Mod Set

The current `load_order.json` enables these four projects, in this order:

1. `t3_enhanced_sfx`
2. `t3_original_soundtracks`
3. `accurate_weapon_damage`
4. `t3_enhanced_t_units`

WARNING `load_order.json` describes the current selection. `Mods.Cod`, `T3_Modded.exe`, and the files under `mods/.compiled/` describe the last build. If the selection changed after that build, those artifacts may be stale. Always run Build Mods + Modded EXE after changing enabled mods or their order.

Quick Glossary

Term	Plain-language meaning
Asset	A game file such as a texture, model, sound, animation, or configuration file.
COD archive	A ZIP-compatible package used by T3 for game data. Important examples are `Game.cod`, `Patch116.Cod`, and `Language.cod`.
Internal path	The path an asset has inside a COD archive, such as `weapons/WM20/weapon1.option`.
Replacement mod	A mod that supplies a new file at the exact internal path of an existing file.
Overlay	A later package temporarily taking priority over an earlier package without deleting the earlier file.
Mod project	A folder containing `mod.json`, `files/`, and optional executable patches.
Build	The process that combines enabled projects into `mods/Mods.Cod` and creates `T3_Modded.exe`.
SCA	A T3 model format used by many static and rigged assets.
LOD	Level-of-detail model data used to swap geometry at different viewing distances.
ANM	A native T3 animation file.
AOB patch	A byte-pattern patch that locates a unique sequence in the original executable before changing it.

Part I - Install and Play Mods Safely

1. What T3-ModTools Does

T3-ModTools 0.6.0 is an experimental Windows application with two modes:

- Asset Extraction reads T3's ZIP-compatible COD archives, copies native assets, and converts supported models into formats that are easier to inspect or edit.
- Modding creates independent projects, manages their order and enabled state, compiles their files into `mods/Mods.Cod`, and builds a separate `T3_Modded.exe`.

The original `T3.exe` is never overwritten. Every build starts from that original executable and produces a new modded executable. This is a central safety feature, not merely a convenience.

The package priority model

When launched through `T3_Modded.exe`, the reference loader gives the generated package priority before the official archives:

```
Highest priority
  mods/Mods.Cod
  Language.cod
  Patch116.Cod
  Game.cod
Lowest priority
```

If `mods/Mods.Cod` contains `weapons/WM20/weapon1.option`, the game reads that version first. The original copy remains inside `Game.cod`. Removing or disabling the mod and rebuilding restores original behavior because the base archive was never edited.

This is why most T3 mods are exact-path replacements. You are not normally editing `Game.cod`; you are placing a higher-priority file at the same internal path.

What the tool can package

The compiler can include native game files while preserving their paths, including:

- textures and skins;
- SCA, LOD, and DET model files;
- WAV and OGG audio;
- ANM animations;
- weapon, character, and vehicle configuration;
- particle, decal, effect, and shader data;
- map files and related configuration.

Replacing an existing asset is the easiest kind of mod. Adding a completely new playable character, weapon, vehicle, or map can also require identifiers, menus, AI definitions, collision data, mission references, and executable research. Begin with replacement work.

2. Install T3-ModTools

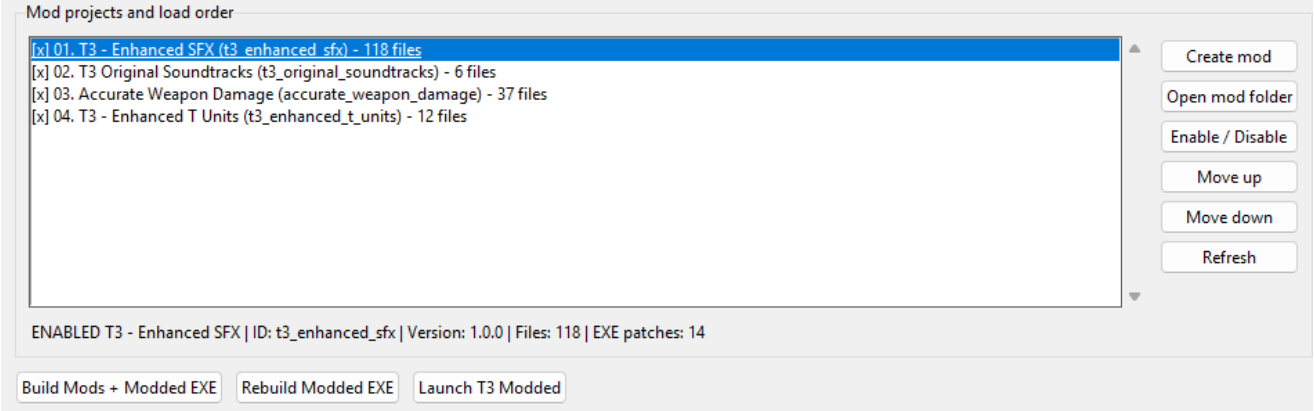
Requirements

- Windows 10 or Windows 11 for T3-ModTools.
- Python 3.9 or newer with Tkinter, which is included in the standard Windows Python installer.
- A legally owned PC copy of Terminator 3: War of the Machines.
- Blender only for optional model and animation work.

T3-ModTools itself does not require third-party Python packages.

Installation steps

1. Extract the entire `T3-ModTools` folder to a normal writable location. Do not run only one script from inside a ZIP file.
2. Double-click `T3-ModTools.bat` to start the graphical interface.
3. Select the Modding mode and Browse your /T3 root game folder. This will automatically create a ‘mods’ folder in that directory.
4. Place your mods in ‘mods’ folder and click ‘refresh’ on the T3-ModTools. This will load a list of your current mods.



5. In order to compile the mods into Mods.Cod, click on ‘Build Mods + Modded EXE’.
6. Once Finished this file should be created in your /T3 Root directory: T3_Modded.exe.
7. Done. To play with mods you should open the game with this launcher. You can either Launch T3_Modded.exe from the root game directory or the T3-ModdingTool clicking on Launch T3 Modded.

WARNING Mods that modify the Modded Launcher will cause the game to crash the first time the modifying mod is compiled. After the initial crash, a new game instance opens automatically, and the game starts without issues. This will not happen again unless a new mod that modifies the launcher is installed.

The program remembers the selected language, mode, archive path, extraction folder, game folder, and mods folder. Settings are stored in:

```
%APPDATA%\T3-ModTools\settings.json
```

Deleting that settings file resets remembered paths without deleting game data or mod projects.

3. Install the Template Mods

Each player should build their own `T3_Modded.exe executable` from the T3-ModdingTools.

Expected game layout

```
T3 game folder/
  T3.exe
  Game.cod
  Language.cod
  Patch116.Cod
  mods/
    t3_enhanced_sfx/
    t3_original_soundtracks/
    accurate_weapon_damage/
    t3_enhanced_t_units/
```

Each project folder must contain its own `mod.json` and `files/` folder. Do not add an extra wrapper level. A common mistake looks like `mods/DownloadedPack/t3_enhanced_sfx/mod.json`; the correct path is `mods/t3_enhanced_sfx/mod.json`.

Installation steps

1. Close T3, 7-Zip, archive preview windows, and any program currently inspecting `Mods.Cod` or `T3_Modded.exe`.
2. Back up the entire game folder or at least `T3.exe` and the current `mods/` folder.
3. Copy the four project folders into `<game folder>/mods/`.
4. Start T3-ModTools and switch to Modding.
5. Select the folder that directly contains `T3.exe`.
6. Confirm that the mods folder is `<game folder>/mods/`.
7. Refresh the project list if the folders are not shown immediately.
8. Enable the four reference projects and arrange them in the order shown below.

Order	Project ID	Main purpose
1	`t3_enhanced_sfx`	Weapon, character, and vehicle sounds plus blue plasma projectile, muzzle, and impact work.
2	`t3_original_soundtracks`	Six music replacements under `Musics/`.
3	`accurate_weapon_damage`	Selected weapon balance changes and translated option-file comments.
4	`t3_enhanced_t_units`	T-900 third-person models, LODs, material values, a Heavy texture, and menu models.

1. Click Build Mods + Modded EXE.
2. Read the result. A successful build creates `mods/Mods.Cod`, `T3_Modded.exe`, and metadata under `mods/.compiled/`.
3. Click Launch T3 Modded, or launch `T3_Modded.exe` directly from the game folder.

IMPORTANT Do not launch `T3.exe` when you expect the generated `Mods.Cod` to load. `T3.exe` remains the unmodified fallback. Use `T3_Modded.exe` for the compiled setup.

4. Understand Load Order and Conflicts

Inside the mod manager, later enabled projects have higher priority when two projects provide the same internal path. Think of the visible list as layers being applied from top to bottom. The last enabled provider wins.

For ordinary asset conflicts, the build records which mod supplied the winning file. This is useful when a texture or configuration change appears to be ignored.

Executable patches are stricter. If two enabled manifests try to write different bytes to the same executable location, the build stops instead of silently choosing one. Identical writes may coexist.

load_order.json

The file stores the project IDs and enabled states:

```
{
  "format_version": 1,
  "mods": [
    {"id": "t3_enhanced_sfx", "enabled": true},
    {"id": "t3_original_soundtracks", "enabled": true},
    {"id": "accurate_weapon_damage", "enabled": true},
    {"id": "t3_enhanced_t_units", "enabled": true},
    {"id": "enhanced_textures", "enabled": false},
    {"id": "dual_wm20", "enabled": false}
  ]
}
```

Use the graphical manager for routine changes. Manual edits are possible, but a typo in an ID can make a project appear missing or move it to a default position.

During a build, T3-ModTools snapshots the current project list, removes stale generated state, and writes a fresh `load_order.json`. It also rebuilds both `Mods.Cod` and `T3_Modded.exe` as one recoverable operation. If either half fails, the tool attempts to restore the previous generated pair so Launch does not mix artifacts from different builds.

5. Verify the Installation In Game

Test visible, audible, and gameplay changes separately. A useful first pass is:

1. Start `T3_Modded.exe` from the game folder.
2. Enter a local match where a T-900, WM20, Termcannon, and the edited units are easy to access.
3. Confirm that the title music or level music includes the replacement tracks when the corresponding cue plays.
4. Fire the WM20 and Termcannon. Check firing sound, muzzle color, projectile color, and surface impact color.
5. Inspect character's third-person models at near and far distances.
6. Inspect the relevant T-900 menu models.
7. Exit the game before rebuilding.

Keep a short test log. Record the build date, enabled mods, order, map, class, weapon, and result. Old games often cache state or fail in ways that look unrelated to the last edit; a written baseline saves time.

6. Update, Disable, or Remove Mods

Disable a project

1. Close the game.
2. Clear the project's Enabled checkbox in Modding mode.
3. Run Rebuild Modded EXE.
4. Run Build Mods + Modded EXE again.
5. Launch the new `T3_Modded.exe` and retest.

Remove a project

1. Disable it and rebuild once if you want a clean comparison.
2. Move the project folder out of `mods/` to a backup location.
3. Refresh the project list.
4. Rebuild.

Update a project

1. Back up the old project folder.
2. Replace the project files, keeping the same ID if it is a normal update.
3. Verify the enabled state and order.
4. Rebuild. Never assume an old `Mods.Cod` automatically includes new project files.

Return to the original game

Launch the untouched `T3.exe`. You may also disable all projects and build an empty mod package, but the original executable is the simplest baseline.

Part II - Extract and Understand Game Assets

7. Asset Extraction Mode

Asset Extraction is the discovery side of T3-ModTools. It lets you find an original file, learn its internal path, and obtain either a native copy or an editable conversion.

Basic extraction

1. Start T3-ModTools and select Asset Extraction at the top of the window.
2. Use the input selector to choose `Game.cod` from the game folder.
3. Choose a new, empty output folder outside the game installation.
4. Select only the category you need for the first pass.
5. Choose conversion options appropriate to that asset type.
6. Click Extract / Convert.
7. Read the final log and inspect the output folders.

The supported categories are Characters, Vehicles, Weapons, Maps, Menu Models, Effects, Audio, and Animations.

Base archive versus patched view

Selecting `Game.cod` gives you that archive's contents. Some official changes live in `Patch116.Cod`, and localized resources can live in `Language.cod`. When the same internal path exists in more than one official archive, the later official package is the version the normal game sees.

For a beginner replacement, begin with `Game.cod`, then check `Patch116.Cod` for the exact same path before editing. T3-ModTools can also read a compatible COD archive directly. Advanced command-line use can point at the installed game folder to build a combined view of its archives.

> **TIP** Always write down the exact archive and internal path that supplied your source. Two files with the same name can have different content or purpose.

8. Choose the Right Output

Native files

Native output is what ultimately belongs in a mod's `files/` folder. Examples include `.tga`, `.wav`, `.ogg`, `.option`, `.dat`, `.sca`, `.lod`, and `.anm`.

Converted output is a working copy for external tools. It is not automatically a valid game asset. An OBJ or glTF file must be exported back into a T3 format before the game can use it.

Static model conversion

Supported static SCA, LOD, and DET content can be converted to OBJ and MTL. OBJ is practical for geometry inspection and simple static editing, but it does not preserve a complete T3 rig or all engine-specific data.

Rigged glTF 2.0

Compatible character, vehicle, and weapon rigs can be exported to glTF 2.0. A rigged export includes:

- `model_rigged.gltf`;
- `model_rigged.bin`;
- original textures when available;
- generated PNG versions of supported DDS or TGA base textures connected to glTF materials.

UV0 is preserved as `TEXCOORD_0`. More than four vertex influences are reduced to the four strongest weights and normalized because that is the base glTF skinning layout.

Animations

The Animations category extracts original `.anm` files under `animations/raw/` and copies the Blender ANM importer into the animation output. Animation files only make sense with a compatible rig. Third-person character animations normally belong to third-person character rigs, while first-person weapon and arm animations belong to first-person rigs.

9. Axes, Mirroring, UVs, and Winding

Coordinate conversion is one of the most common reasons a model looks mirrored, sideways, invisible, or inside out.

Extraction conventions

- All converted models receive a global X mirror.
- Characters, vehicles, maps, and menu models otherwise keep the game's native Y/Z orientation.
- Weapons and effects may optionally use Z-up.
- The OBJ option Flip UV V coordinate affects OBJ output only.
- Rigged glTF always preserves the source V coordinate, regardless of that OBJ checkbox.

Practical rule

Do not apply extra flips by instinct. Record the extraction settings and let the matching exporter reverse them. Current LOD import/export plug-ins store and reuse import settings. For a glTF produced by T3-ModTools, the general SCA exporter normally uses Undo T3-ModTools X mirror.

Face winding determines which side of a triangle is visible. A mirror changes the transform determinant and can reverse winding. The current LOD and menu exporters account for the determinant. If a model disappears from one side, verify transforms and exporter settings before manually flipping every face.

UV rule

An upside-down texture usually means the V coordinate was flipped at the wrong stage. For OBJ, use the extraction checkbox consistently. For rigged glTF, do not add an OBJ-style V flip. Menu models use their specialized exporter, which preserves UV V according to the verified menu-model rules.

10. Find the Internal Path That Matters

A mod only works when its file lands at the path the game actually requests. Use this process:

1. Extract the smallest relevant category.
2. Search filenames and text references for the character, weapon, texture, or sound name.
3. Inspect small `.dat`, `.option`, `.lib`, `.sca`, and material files in a text editor.
4. Follow referenced filenames. A model may name a texture, a sound definition may name a WAV, and an animation library may name an ANM.
5. Check whether the same path appears in `Patch116.Cod`.
6. Reproduce the final path under your mod's `files/` folder.

Examples:

Goal	Internal path example
Edit WM20 primary gameplay	`weapons/WM20/weapon1.option`
Replace WM20 fire definition	`Sound44/Sound/Weapons/wm20/weapon.dat` plus matching banks
Replace T-900 Supply third-person model	`CA/casts/t900s/tps/model.sca` and `model.lod`
Replace a music cue	`Musics/palya5.ogg`
Edit traveling plasma	`Particles/FiredBullet.dat`
Edit plasma muzzle	`Particles/MuzzleFire.dat`

Preserve the spelling used by the archive, including native tokens such as `granade`. Although the compiler detects path conflicts case-insensitively, matching the original capitalization and path is the safest habit.

Part III - Create, Build, and Test Your Own Mod

11. Create a Mod Project

Use the graphical manager

1. Close the game.
2. Open T3-ModTools and switch to Modding.
3. Select the game folder containing `T3.exe`.
4. Click Create mod.
5. Enter a clear display name, author, and short description.
6. Let the tool create the safe project ID and folder structure.
7. Select the new project and click Open mod folder.

The canonical structure is:

```
mods/
  my_first_mod/
    mod.json
    files/
    patches/
```

An example `mod.json` is:

```
{
  "format_version": 1,
  "id": "my_first_mod",
  "name": "My First Mod",
  "author": "Your Name",
  "version": "1.0.0",
  "description": "A small exact-path replacement mod."
}
```

Keep the folder name and `id` stable after release. The name is for people; the ID is what `load_order.json` uses.

12. Make a First Exact-Path Replacement

A texture replacement is a good first project because it teaches the complete loop without executable work.

1. Extract the relevant Characters, Weapons, Vehicles, or Menu Models category.
2. Identify one native texture and its exact internal path.
3. Copy the original to a separate work folder.
4. Edit the copy without changing its dimensions or format on the first attempt.
5. Recreate the internal directories under your project:

```
my_first_mod/
  files/
    CA/
      textures/
        example.tga
```

1. Place the edited native texture at that path.
2. Enable only this project for the first test.
3. Click Build Mods + Modded EXE.
4. Launch the modded game and inspect the asset under predictable lighting.
5. If it works, add a second change. If it fails, compare the path, filename, format, and source archive before changing anything else.

WARNING Do not put a ZIP containing the real replacement tree inside `files/`. The compiler preserves that ZIP as one file; it does not unpack nested archives. Put the native replacement files themselves under `files/`.

13. Build Discipline and Versioning

One change per test

When learning an undocumented engine, a build with ten unrelated edits creates ten possible causes. Prefer small, named milestones:

- `1.0.0` first working replacement;
- `1.0.1` path or packaging correction;
- `1.1.0` new asset family;
- `2.0.0` incompatible redesign.

Keep source and release files separate

Your project `files/` folder should contain only game-ready native assets. Keep Blender files, PSD files, high-resolution masters, audio project sessions, notes, and extracted originals in a separate source workspace.

Distribution checklist

- Include the project folder with `mod.json`, `files/`, and optional `patches/`.
- Include a readme with requirements, install order, conflicts, and test notes.
- Do not redistribute original extracted game archives or unrelated assets.
- Do not distribute your generated `T3_Modded.exe` as the preferred installation path.
- Tell users to build from their own original executable.
- Name any required T3-ModTools version.
- State whether the project has executable patches.

14. Troubleshoot the Build

Access denied or WinError 5

The usual cause is a locked file.

1. Close T3 and wait for it to exit completely.
2. Close 7-Zip, Explorer archive previews, hex editors, antivirus scans, or other tools viewing `Mods.Cod` or `T3_Modded.exe`.
3. Confirm that the game and mods folders are writable.
4. Run the build again.
5. If security software quarantines a newly generated executable, restore or allow only the local tool and file you trust. Do not disable protection globally.

A project does not appear

- Confirm `mod.json` is directly inside the project folder.
- Validate the JSON syntax.
- Confirm the folder is under the selected mods directory.
- Make sure the folder name does not begin with a dot.
- Refresh the project list.

The game launches but the mod is absent

- Confirm you launched `T3_Modded.exe`.
- Rebuild after the last enabled-state or order change.
- Check `mods/.compiled/build_manifest.json` for the file and winning project.
- Compare the internal path with the extracted source.
- Check whether a later enabled mod replaces the same path.
- Verify that the file is a native game format, not an editing format.

The build stops on an executable patch

- The AOB pattern may have zero or multiple matches in this executable version.
- The replacement may not be the same length for `aob_replace`.
- An `aob_write` destination may fail its expected-byte validation.
- Two enabled mods may write different bytes to the same location.
- Restore a pristine original `T3.exe` and avoid using a previously modified executable as the source.

Part IV - Workshops Based on the Active Mods

15. Workshop: Weapon and Vehicle Sound Replacement

The active `t3_enhanced_sfx` project demonstrates two audio strategies: replacing a WAV at an existing path, and editing a text sound definition so it points to a different WAV.

Understand the three sound banks

T3 carries parallel trees named `Sound11`, `Sound22`, and `Sound44`. The reference mod repeats relevant definitions and WAV files in all three trees so the replacement remains available across game audio configurations.

Do not assume the folder name will convert your audio automatically. The current WM20 and Termcannon replacement WAVs are mono, 16-bit, 44.1 kHz files repeated in all three banks.

Replace a weapon firing sound

For the WM20, the definition path is repeated as:

```
Sound11/Sound/Weapons/wm20/weapon.dat
Sound22/Sound/Weapons/wm20/weapon.dat
Sound44/Sound/Weapons/wm20/weapon.dat
```

The active definition changes the FIRE event from `plasma_04_jo.wav` at volume 180 to `t3-wm20.wav` at volume 255. The WAV lives beside the weapon's other sound files inside each bank.

Use this method:

1. Extract Audio and find the target weapon's `weapon.dat`.
2. Read the event blocks and note the relative WAV path.
3. Prepare a mono PCM WAV. For the first test, match the original sample format and keep the peak below clipping.
4. Give the file a unique, simple ASCII filename.
5. Update only the intended event's `filename` and, if necessary, `volume`.
6. Repeat the edited definition and WAV under `Sound11`, `Sound22`, and `Sound44`.
7. Preserve every other parameter until the new sound is confirmed.
8. Build and test single shots, rapid fire, indoor spaces, and several simultaneous actors.

The Termcannon follows the same pattern. Its active FIRE event points to `termcannon-v3.wav` at volume 255 instead of `plasma_single_v2.wav` at volume 245.

Automatic-fire tails and overlap

If a long tail is baked into every automatic shot, repeated events can overlap and become too loud or muddy. Current research has not established a universal per-actor single-voice rule for all T3 weapons. Treat sound design as the first solution:

- keep the repeated shot body short enough for the firing interval;
- use a restrained tail;
- test several actors firing together;
- avoid normalizing every sample to maximum loudness;
- compare in game, not only in an audio editor.

Replace a looping vehicle sound

The HK example edits `Sound/Vehicles/HK/hk.dat` inside each sound bank. It points the near and distant loops to `hk_idle.wav` and `hk_idle_dist.wav` instead of the calculated originals.

For a loop:

1. Match the original channel count and format.
2. Edit at zero crossings or use a clean crossfade.
3. Keep the distant version simpler and quieter than the near version.
4. Preserve the definition's distance and behavior parameters during the first test.
5. Test approach, pass-by, idle, and departure transitions.

16. Workshop: Replace Music

The active `t3_original_soundtracks` project is a clean exact-path replacement mod. It supplies six OGG files:

```
Musics/palya4.ogg
Musics/palya5.ogg
Musics/palya6.ogg
Musics/palya8.ogg
Musics/palya10.ogg
Musics/palya13.ogg
```

This is a good beginner project because no executable patch or registration file is needed when an existing cue is replaced.

Procedure

1. Extract Audio and locate the original OGG cue.
2. Copy the original to a backup outside the mod.
3. Export the replacement as OGG Vorbis.
4. Match the exact filename and internal path.
5. Start with a comparable channel layout, sample rate, duration, and loudness.
6. Place the file under `files/Musics/` in the mod project.
7. Enable only the music project for the first test.
8. Trigger the exact level or menu state that uses the cue.
9. Listen for start delay, abrupt ending, looping gap, distortion, and loudness relative to effects.

Replacing a file does not change when the engine chooses that cue. Changing cue selection or adding a new music slot requires following the game's configuration references and may require further research.

17. Workshop: Edit Weapon Balance

The active `accurate_weapon_damage` project demonstrates editable text configuration under `weapons/`. Most included files preserve original gameplay values while translating comments into English. Four option files currently change gameplay values.

Fields commonly found in weapon1.option

Field	Observed role	Editing advice
`weight`	Equipment weight.	Change only after confirming class/loadout effects.
`magazineSize`	Ammunition loaded before a reload; zero can have special behavior.	Test reload and HUD count.
`bullets`	Reserve ammunition outside the magazine.	Test spawn, pickup, and resupply.
`shootsPerMin`	Engine fire-rate parameter. Values do not resemble ordinary RPM in every file.	Treat as a game-specific parameter and test direction and timing.
`type`	Projectile family, such as `bullet`, `granade`, `missile`, or `homingMissile`.	Preserve native spellings exactly. A type change can alter more than damage.
`subType`	Weapon-category index used by other systems.	Do not change casually; particles and executable routing can depend on it.
`range`	Observed minimum and maximum effect or blast range.	Test several surfaces and targets.
`damage`	Observed minimum and maximum damage or effect strength.	Change gradually and compare target types.
`speed`	Initial projectile speed.	Test collision, visual trail, and aiming behavior.
`throw`	Spread angle in degrees according to the translated source comments.	Test standing, moving, and sustained fire.
`timeToExplode`	Immediate, delayed, or sticky countdown behavior depending on sign.	Use the original token pattern as a template.
`MRV`, `TETA`, `DMP`	Turning, inertia, and braking parameters in guided or mounted weapon contexts.	Edit one at a time.
`heatUp`	Heat or warm-up behavior.	Test sustained fire and cooldown.

Confirmed changes in the active project

File	Base value	Active value
`weapons/HK_Tank/weapon1.option` and `weapon2.option`	`shootsPerMin 8`, `type missile`, `damage 0.0 0.125`, `heatUp 8 1.0`	`shootsPerMin 4`, `type granade`, `damage 0.0 0.25`, `heatUp 2 1.0`
`weapons/M60_60_Stand/weapon1.option`	`damage 0.125 0.125`	`damage 0.25 0.25`
`weapons/Termcannon/weapon1.option`	`damage 0.0 0.2`	`damage 0.0 0.4`

The remaining option files in the active project currently keep the same parsed gameplay values and mainly translate comments. This distinction matters when documenting the mod.

Build a small balance mod

1. Extract Weapons.
2. Copy one target `.option` file to a separate work folder.
3. Create a new mod project.
4. Recreate the path under `files/weapons/`.
5. Change one numeric field by a modest amount.
6. Preserve braces, native key names, spelling, and number order.
7. Build with only your balance mod enabled.
8. Test against a repeatable target and record shots to kill, timing, blast behavior, and HUD ammo.
9. Restore the original value before testing a different field if the first result is unclear.

TIP The token `granade` is misspelled in the native data. Keep the engine's spelling. Correcting it to `grenade` can turn a readable file into an invalid configuration.

18. Workshop: Plasma Projectile, Muzzle, and Impact Effects

The visible shot is not one effect. T3 separates at least three stages:

Stage	Reference data	What it controls
Muzzle	`Particles/MuzzleFire.dat`	Front and side muzzle textures plus muzzle light color.
Traveling projectile	`Particles/FiredBullet.dat`	Side trail and front-facing projectile textures.
Surface impact	`Particles/BulletHit.dat`, `BulletHitPlasma.dat`, and the added blue bank	Impact particles selected by weapon subtype and surface.

Changing only `FiredBullet.dat` can produce a blue bolt with a red muzzle and red impact. Test all three stages separately.

Make the WM20 and Termcannon projectile blue

1. Extract Effects.
2. Open `Particles/FiredBullet.dat` in a text editor.
3. Find the blocks for `Weapon_Wm20` and `Weapon_TermCannon`.
4. Change their side texture from the red plasma texture to a blue trail texture.
5. Change their front texture to a blue plasma front texture.
6. Add the referenced TGA files under `files/Textures/Particles/` at their exact paths.
7. Repeat the weapon-specific edits in `Particles/MuzzleFire.dat`.
8. For the active blue light, the tested RGB values are `0.59`, `0.67`, and `1`.
9. Build and test the muzzle and projectile before attempting impacts.

Why a new blue impact DAT is not enough

The original executable initializes a limited set of global bullet-impact banks. Creating `Particles/BulletHitPlasmaBlue.dat` does not automatically register or select it.

The active SFX project solves the specific WM20 and Termcannon case with an executable patch that:

- allocates and initializes an independent blue `cBulletHit` bank;
- loads `Particles/BulletHitPlasmaBlue.dat` for that bank;
- routes subtype 1, WM20, and subtype 15, Termcannon, to blue impact IDs;
- preserves the original red plasma bank for other weapons;
- draws, updates, and releases both banks;
- preserves the twelve surface variants used by the impact system.

This is a verified case-specific solution, not a general automatic particle-registration feature. Beginners should reuse the working project as a dependency or study template. Do not paste its byte sequences into another executable version without validation.

19. Workshop: T-900 Models, Textures, Materials, and Menus

The active `t3_enhanced_t_units` project demonstrates that a character appearance can span several asset systems.

Third-person model pair

The Supply and Heavy third-person replacements include both files:

```
CA/casts/t900s/tps/model.sca
CA/casts/t900s/tps/model.lod
CA/casts/t900heavy/tps/model.sca
CA/casts/t900heavy/tps/model.lod
```

Including only `model.sca` is not a complete character replacement. The game can switch to `model.lod` based on distance or code path, leaving the original or a mismatched model visible. Treat SCA and LOD as a pair.

The active Supply SCA references `t900.tga`. The active Heavy SCA references a separate `t900_heavy.tga`, allowing the Heavy unit to use a distinct body texture instead of sharing the Supply texture.

Both active third-person SCA files use:

```
Shader
{
    Type DIFFUSE_SPECULAR
    BaseTexture "..."
}
```

They contain the original-compatible 32-bone hierarchy. Bone names and hierarchy are part of animation compatibility, not cosmetic labels.

Material files

T3's `.mat` files are small legacy material parameter files, not modern physically based materials. Blender roughness and metallic sliders are not automatically translated.

The active project currently uses:

```
CA/textures/t900.mat        -> 1 2 18
CA/textures/t900_heavy.mat  -> 1 2 18
CA/textures/017.mat         -> 1 12 18
```

These are empirical project values. Do not assume that every model needs the same triplet or that each number maps cleanly to a modern PBR property. Start from the target asset's original `.mat`, change one value, and compare under several maps and lighting conditions.

Menu models are a separate pipeline

The active project also replaces:

```
MenuDatas/3DModells/t900_posed.sca
MenuDatas/3DModells/t900_supply.sca
MenuDatas/3DModells/t900_s_illum.tga
MenuDatas/3DModells/t900_s_illum2.tga
```

Menu models use verified rules that differ from world character SCA files:

- texture library `MenuDatas/3DModells/`;
- shader type `BASE`;
- native menu-model axes;
- global X mirror undone;
- UV V preserved;
- determinant-aware winding.

Use `T3_Blender_Menu_Model_SCA_Exporter.py` for these assets. Do not use world-model settings and then repair the result with arbitrary rotations.

Recommended model test matrix

1. Test Supply and Heavy separately.
2. Test front, back, and profile views.
3. Test idle, walk, run, fire, hit, and death animations.
4. Test near, medium, and far camera distances.
5. Test bright human maps and red Terminator vision.
6. Test the menu selection and loadout screens.
7. Watch for missing faces, inverted normals, stretched UVs, wrong bone weights, and sudden LOD shape changes.

Part V - Blender Workflows

20. Install the Blender Plug-ins

The current T3-ModTools Blender Plugins folder includes SCA, LOD, menu-model, ANM, and bone-frame tools. Install only the plug-ins needed for the current task.

1. Open Blender.
2. Choose Edit > Preferences > Add-ons.
3. Click Install from Disk.
4. Select one `.py` plug-in file.
5. Enable the added plug-in.
6. Repeat for other required plug-ins.

Safe filenames do not contain dotted version suffixes. A filename such as `tool_v0.1.0.py` can be interpreted as a nested module path by some Blender versions. Keep the distributed simple filenames.

Plug-in	Use
`T3_Blender_SCA_Exporter.py`	Export a selected static or rigged replacement to SCA.
`T3_Blender_LOD_Importer.py`	Import a native model LOD template with ranges, materials, UVs, and weight maps.
`T3_Blender_LOD_Exporter.py`	Replace imported LOD levels while preserving template data.
`T3_Blender_Menu_Model_SCA_Exporter.py`	Export menu SCA files using specialized menu rules.
`T3_Blender_ANM_Importer.py`	Import an ANM onto a compatible armature.
`T3_Blender_ANM_Exporter.py`	Export the active Action back to ANM.
`T3_Frame_Per_Bone_exporter.py`	Analyze and apply frame-to-frame bone component deltas.

These plug-ins are experimental. Keep the extracted original beside every exported replacement and test the result as a mod before overwriting any work file.

21. Rigged SCA Replacement Workflow

Import the working rig

1. In Asset Extraction, select Characters and enable rigged glTF export.
2. Locate `model_rigged.gltf` under `converted_rigged/`.
3. In Blender, use File > Import > glTF 2.0.
4. Preserve the armature, bone names, and hierarchy.
5. After importing a rigged character model produced by T3-ModTools, select all character mesh objects and mirror them on the global Y axis.
6. Select the armature's bones and mirror them separately on the global Y axis.
7. Confirm that the mesh and skeleton are aligned before editing or exporting. The required order is mesh first, bones second.

Edit safely

- Apply required mesh modifiers deliberately.
- Triangulate or validate the final mesh.
- Keep vertex weights to the target bones.
- Avoid renaming bones.
- Keep bounding proportions plausible for the original collision and animations.
- Use separate materials only when the target format and path references support them.

Export

1. Select the mesh objects. Select or link the compatible armature through an Armature modifier.
2. Use File > Export > T3 Model (.sca).
3. Set the internal texture library, for example `CA/Textures`.
4. Keep Undo T3-ModTools X mirror enabled for a mirrored glTF source whose transforms were applied.
5. Export to a work folder.
6. Inspect the SCA text before packaging.
7. For the active T-900 third-person target, verify `BoneNumber 32`, the expected bone names, `Type DIFFUSE_SPECULAR`, and the intended base texture. The current general exporter may require target-specific shader verification rather than blind acceptance of a default.
8. Put the final file under the exact `CA/casts/.../tps/model.sca` path.
9. Export or preserve a compatible `model.lod` too.

Practical geometry limits

Native submesh indices are effectively 16-bit, so a submesh cannot address more than 65,536 stored vertices. The current text exporter can duplicate vertices per triangle corner. In that worst case, the practical ceiling is about 21,845 triangles per material submesh.

Recommended starting budgets for a character project are:

Level	Suggested triangle range	Purpose
LOD0	25,000 to 35,000	Near view, if exporter submesh limits are respected.
LOD1	12,000 to 18,000	Medium-near.
LOD2	5,000 to 8,000	Medium-far.
LOD3	1,000 to 2,000	Far view.

These are recommendations, not engine guarantees. Split materials and submeshes carefully and test performance on the original game's intended scale.

22. LOD Import and Export

The LOD exporter is template-based. This is valuable because a native `model.lod` can contain bones, instances, collision targets, and unedited levels that should not be rebuilt from guesses.

How LOD levels work

LOD means level of detail. A model.lod defines how the same character is shown at different viewing distances. LOD0 is the version used when the character is near the camera and normally has the most geometry and visual detail. As the LOD number increases, quality and polygon count should decrease because those versions are used farther away.

Workflow

1. Install both LOD plug-ins.
2. Import the original `model.lod` with File > Import > T3 LOD.
3. For a rigged .lod import, select only the bones and mirror them on the global Y axis. Do not mirror the LOD mesh as part of this correction.
4. Keep the custom properties created by the importer.
5. Edit the intended LOD objects or generate lower levels with controlled Decimate modifiers.
6. Use File > Export > T3 LOD.
7. Select the original `model.lod` as the template.
8. Keep Export all LODs from the imported set enabled when the complete set is present.
9. Keep Preserve missing LODs from template enabled unless you intentionally replace every level.
10. Keep Use settings stored by T3 LOD Importer enabled for a direct import/export round trip.
11. The normal direct-import defaults convert Blender Z-up back to T3 Y-up and flip UV V back according to stored settings. Undo X mirror is normally disabled for direct LOD imports, but may be needed for geometry originating from a mirrored T3-ModTools glTF.
12. Export, package both SCA and LOD, and test every distance transition.

The original T-900 LOD ranges observed in the reference asset were approximately:

Level	Original triangle count	Distance range
LOD0	5,185	0 to 10
LOD1	3,637	10 to 20
LOD2	2,208	20 to 50
LOD3	831	50 to 1000

Use the target file's own ranges as the first template. A higher-detail replacement can need different budgets, but arbitrary distance changes can create popping or performance problems.

23. Animation Import and Export

Import

1. Extract a compatible rigged glTF model and the target ANM.
2. Import the glTF into Blender.
3. Follow the current importer workflow for the model and armature orientation, including the documented global Y inversion when required by that workflow.
4. Select the armature.
5. Use File > Import > T3 Animation (.anm).
6. Read the unmatched-bone report.

The importer stores the source FPS, `MaxTimeValue`, complete joint paths, and exact joint order on the generated Action.

Edit

- Keep the imported Action metadata.
- Preserve the source joint set for an overlay animation.
- Avoid adding unrelated armature bones to a partial upper-body action.
- Check root motion and rest pose before changing timing.
- Work on a duplicate Action.

Export

1. Make the edited Action active on the compatible armature.
2. Use File > Export > T3 Animation (.anm).
3. Keep Preserve imported ANM joint set enabled.
4. If the Action was not created by the importer, select a native reference ANM with the same role.
5. Place the exported ANM at the original internal path under the mod's `files/` folder.
6. Build and test transitions into and out of the animation, not only the isolated loop.

The exporter samples every integer frame in the Action range. Constraints are evaluated through final pose matrices, but game-specific animation events are not generated automatically.

24. Bone Frame Delta Tool

`T3_Frame_Per_Bone_exporter.py` provides a specialized Blender panel under View3D > Sidebar > Animation > Bone Frame Delta. It can export pose-bone component values and frame-to-frame deltas, then reapply edited delta reports.

This is useful for understanding or adjusting an animation numerically without treating every frame as unrelated. Rotation components use quaternions in WXYZ order. Because quaternions `q` and `-q` represent the same rotation, sign continuity matters when calculating deltas. The tool is designed to avoid false jumps caused only by that equivalent sign change and to tolerate near-zero floating-point noise.

Use it as an advanced diagnostic aid:

1. Duplicate the Action.
2. Export a short frame range for one or two bones.
3. Edit one clearly identified component delta.
4. Apply it to the duplicate Action.
5. Inspect the curve and viewport motion.
6. Export to ANM only after the edited Action remains stable.

Do not use a numerical delta report as a substitute for rig compatibility or a correct rest pose.

Part VI - Executable Patches and Advanced Research

25. Executable Patch Manifests

Ordinary asset replacement should always be preferred when it can solve the problem. Executable patches are for behavior or registration that data files alone cannot reach.

The canonical path is:

```
my_mod/
  patches/
    executable.json
```

A flat `my_mod/executable.json` is accepted as a compatibility fallback and is used by the current enhanced SFX project.

aob_replace

```
{
  "format_version": 1,
  "target": "T3.exe",
  "patches": [
    {
      "id": "example_patch",
      "description": "Describe the behavior change",
      "type": "aob_replace",
      "section": ".text",
      "pattern": "8B 45 ?? 83 F8 02",
      "replacement": "8B 45 ?? 83 F8 03",
      "expected_matches": 1
    }
  ]
}
```

`??` in a pattern matches any byte. `??` in a replacement preserves the original byte. Pattern and replacement must have the same length.

aob_write

`aob_write` finds a unique anchor, validates destination bytes, and writes at a relative offset. It is used for controlled code or data injection away from the anchor.

```
{
  "id": "example_injection",
  "description": "Write validated bytes near a unique signature",
  "type": "aob_write",
  "section": ".text",
  "pattern": "AA BB CC DD",
  "write_offset": "0x20",
  "expected_fill": "00",
  "replacement": "11 22 33 44",
  "expected_matches": 1
}
```

Safety rules enforced by the tool

- Patterns are searched against the pristine original `T3.exe`, not a previous modded executable.
- Version 1 requires exactly one expected match.
- Searches can be limited to `.text`, `.rdata`, `.data`, `.tls`, `.data1`, `.rsrc`, or `all`.
- `aob_write` must remain in the selected section.
- Expected bytes or fill are validated before writing.
- Conflicting writes stop the build.
- `T3.exe` is never overwritten.

An AOB patch is not automatically portable across releases, cracks, localized executables, or prior binary modifications. If a pattern does not match exactly once, stop and research that executable rather than weakening validation.

26. Known Limits and Research Frontiers

New content registration

File replacement is well understood. New character, weapon, vehicle, map, particle bank, or menu slots can require configuration and code paths not yet generalized by the tool.

Sound concurrency

Long automatic-weapon tails can overlap. A general rule for limiting every weapon to one active voice per actor has not been established. Design short repeated layers and test concurrency.

Terminator vision gamma

The original manual assigns Q to binocular or night vision. T3's Terminator presentation also interacts with legacy gamma behavior and `SkyNet_Gamma` settings. On modern Windows, the world tint can fail while HUD overlays remain visible. First try the game's Video options, toggle and reapply the Terminator screen option, use exclusive fullscreen, and disable external HDR, Auto HDR, night-light, or color utilities for the test. A robust modern shader replacement remains separate research.

Model export coverage

The general SCA exporter does not automatically generate map binary trees, `scene.det`, `scene.dyn`, collision targets, advanced shader parameters, lightmaps, or gameplay registration. LOD export is a separate template-based workflow.

Animation coverage

Individual animations can still require target-specific axis, root-motion, or rest-pose correction. Events tied to gameplay are not generated simply by exporting pose frames.

Renderer and shader modernization

Replacing textures and legacy material values is practical today. A generalized modern renderer, physically based material conversion, or universal post-processing pipeline is not part of the verified workflow.

Part VII - Reference

27. Folder and Path Reference

```
T3 game folder/
  T3.exe                      Original fallback executable
  T3_Modded.exe               Generated modded executable
  Game.cod                    Base game archive
  Language.cod                Language archive
  Patch116.Cod                Official patch archive
  mods/
    load_order.json           Current visible order and enabled state
    Mods.Cod                  Generated merged package
    .compiled/                Generated build and loader metadata
    my_mod/
      mod.json                Project identity and version
      files/                  Exact internal asset paths
      patches/
        executable.json       Optional AOB patch manifest
```

Common internal roots:

Root	Typical content
`CA/`	Character models, textures, animation libraries, and animations.
`weapons/`	Weapon option data and models.
`vehicles/`	Vehicle data and models.
`Sound11/`, `Sound22/`, `Sound44/`	Parallel sound definitions and WAV assets.
`Musics/`	OGG music cues.
`Particles/`	Particle behavior definitions.
`Textures/Particles/`	Particle textures.
`MenuDatas/3DModells/`	Menu model SCA files and their textures.
`scene/`	Map and scene data.

28. Pre-Build Checklist

- The game is closed.
- No archive tool is holding `Mods.Cod` open.
- The selected game folder directly contains the pristine `T3.exe`.
- The selected mods folder is the intended `<game>/mods/` folder.
- Every project has a valid `mod.json`.
- Every replacement is under `files/` at the exact internal path.
- Editing sources such as `.blend`, `.psd`, and project sessions are outside `files/`.
- Enabled projects and order are correct.
- Experimental projects are disabled unless intentionally tested.
- Executable-patch dependencies and conflicts are understood.
- The last known good project version is backed up.

29. Post-Build Checklist

- The build reports success.
- `mods/Mods.Cod` has a current timestamp.
- `T3_Modded.exe` has a current timestamp.
- `mods/.compiled/build_manifest.json` lists the expected enabled projects.
- The manifest lists the expected winning files.
- `mods/.compiled/loader_manifest.json` lists the expected executable patches.
- The original `T3.exe` remains unchanged.
- The game is launched through `T3_Modded.exe`.
- The test covers the exact asset, several distances or contexts, and a vanilla comparison.

30. Failure-to-Cause Map

Symptom	Most likely checks
Mod is completely absent	Wrong executable, stale build, wrong mods folder, invalid project structure.
One asset is absent	Wrong internal path, later mod conflict, converted file instead of native file.
Near model works, far model is original	Missing or mismatched `model.lod`.
Texture is upside down	UV V flipped at the wrong conversion stage.
Model is mirrored	X mirror reversed twice or not reversed at export.
Faces disappear	Winding changed by a negative determinant, singular transforms, or bad normals.
Character deforms during animation	Bone names, hierarchy, rest pose, or weights do not match.
New impact DAT is ignored	The executable does not register or route to the new bank.
Build reports access denied	Game, archive viewer, editor, or security tool holds an output file.
AOB pattern fails	Different executable build, non-pristine source, or non-unique pattern.
Music never plays	Wrong cue path or the tested game state does not select that cue.
Weapon audio becomes muddy	Repeated sample tail overlap, excessive volume, or too many simultaneous emitters.

31. A Repeatable Modding Method

The most reliable T3 modding loop is simple:

1. Observe one specific behavior.
2. Extract the smallest relevant asset set.
3. Trace the exact internal path and references.
4. Back up the original and create a separate mod project.
5. Change one variable.
6. Build from the current load order.
7. Launch `T3_Modded.exe`.
8. Test in a repeatable scenario.
9. Record the result.
10. Keep the successful version before expanding scope.

This method is slower for one edit and much faster for a complete mod. It separates facts from guesses, turns failures into useful evidence, and makes discoveries reusable by the next person.

Source Notes

This edition consolidates the T3-ModTools 0.6.0 readme, changelog, executable-patch specification, implementation and test reports; the current reference projects and load order; the original game manual; extracted archive structure; Blender plug-in documentation and implementation; and the verified results and unresolved findings recorded during development of the current mods.

The original manual reflects the game's release era. T3-ModTools requirements and modern troubleshooting are separate from the original game's historical system requirements.

T3 Modding Guide - Edition 1.0